Ramdeobaba University, Nagpur

Department of Artificial Intelligence and Machine Learning

CivicSignal

An Anonymous Civic Issue Reporting and AI-Assisted Community Engagement Platform

Project Report submitted in partial fulfillment of the requirements for the Community Engagement Project (CEP)

Submitted by

Abbas Nadir	Roll No. 29
Vedant Kashikar	Roll No. 66
Piyush Khokale	Roll No. 43
Amey Shendre	Roll No. 10
Ayushi Chavhan	Roll No. 6

Guide: [Name of Guide]

Partner NGO: [Name of NGO / Community Organization]

Academic Year: 2025–2026

Month: April 2026

Certificate

This is to certify that the Community Engagement Project titled **CivicSignal: An Anonymous Civic Issue Reporting and AI-Assisted Community Engagement Platform** is a bonafide work carried out by Abbas Nadir (29), Vedant Kashikar (66), Piyush Khokale (43), Amey Shendre (10), and Ayushi Chavhan (6), submitted to Ramdeobaba University, Nagpur in partial fulfillment of the requirements of the Community Engagement Project under the Department of Artificial Intelligence and Machine Learning during the academic year 2025–2026.

NGO Coordinator	**Project Guide**
[Name and Signature]	[Name and Signature]
[Organization Name]	Department of Artificial Intelligence and Machine Learning

Head of Department: Dr. Rashmi Welekar
Place: Nagpur
Date: [To be filled]

Declaration

We hereby declare that the project report titled **CivicSignal: An Anonymous Civic Issue Reporting and AI-Assisted Community Engagement Platform** is our original work carried out in the Department of Artificial Intelligence and Machine Learning of Ramdeobaba University, Nagpur. This work has not been submitted earlier, either in whole or in part, for the award of any degree or diploma at this or any other institution.

Name	Roll No.	Signature
Abbas Nadir	29	
Vedant Kashikar	66	
Piyush Khokale	43	
Amey Shendre	10	
Ayushi Chavhan	6	

NGO Signed Acknowledgement Copy

Attach the signed acknowledgement letter from the NGO or community organization on official letterhead in this section before final submission.

Acknowledgement

We express our sincere gratitude to our project guide, department faculty, and partner NGO for their guidance, feedback, and support during the planning, development, and documentation of CivicSignal. We thank the open-source communities behind Next.js, React, Express, TypeScript, Supabase, Groq, and related tooling that helped us build this project. We also thank our classmates, peers, and families for their encouragement throughout the CEP journey.

Abstract

Urban communities frequently face recurring civic problems such as potholes, drainage failures, broken street lights, sanitation lapses, waterlogging, and encroachment. Existing complaint mechanisms are often fragmented, opaque, and difficult to track. Citizens may also hesitate to report issues because of identity exposure or uncertainty about the correct department or portal.

CivicSignal is a full-stack web platform designed to improve civic reporting, issue visibility, and institutional response. The system allows citizens to register, create anonymous complaints with optional image evidence and location context, browse a ranked public feed, comment on posts, raise issues, follow cases, and report abusive content. The institution side supports NGO staff, government staff, and administrators through a professional dashboard that offers metrics, charts, filtered work queues, case tracking, moderation controls, and role-scoped workflow actions.

The platform is implemented using Next.js 16 and React 19 on the frontend, Express and TypeScript on the backend, and Supabase for authentication, relational storage, and media storage. AI support is integrated through Groq. Groq is currently used for ingestion-time spam filtering and for a signed-in complaint-assistance chatbot that explains app workflows and guides users toward complaint-registration related actions and official grievance channels.

The project combines privacy-aware civic reporting, role-based institutional workflows, and AI-assisted guidance in a single application. It demonstrates how a modern web stack can support community engagement, structured complaint management, and socially relevant software engineering within the CEP framework.

Keywords: civic technology, anonymous reporting, community engagement, institutional dashboard, AI-assisted filtering, Groq, Supabase, Next.js, Express, urban governance

Contents

Certificate **i**

Declaration **ii**

NGO Signed Acknowledgement Copy **iii**

Acknowledgement **iv**

Abstract **v**

1 Introduction **1**

1.1 Overview of the Community Engagement Project 1
1.2 Importance of Digital Solutions for Community Problems 1
1.3 Objectives of the Website 1
1.4 Problem Statement 2
1.5 Scope of the Project 2

2 Community Analysis **3**

2.1 Description of the Target Community 3
2.2 Identified Problems 3
2.3 Need for a Website-Based Solution 3
2.4 Stakeholders Involved 4

3 Literature Review **5**

3.1 Existing Websites and Portals in Similar Domains 5
3.2 Technologies Used in Similar Systems 5
3.3 Limitations of Current Solutions 5

4 System Requirements **7**

4.1 Functional Requirements 7
4.1.1 Citizen Module 7
4.1.2 Institution Module 7
4.1.3 AI and Automation Module 8

4.2 Non-Functional Requirements . . . 8
4.3 Hardware and Software Requirements . . . 8

5 System Design **10**
5.1 Architecture Diagram . . . 10
5.2 Database Design . . . 10
5.3 UML Diagrams . . . 11
5.3.1 Use Case View . . . 11
5.3.2 Sequence View for Post Creation . . . 12
5.4 Wireframes and UI Design . . . 12

6 Methodology **13**
6.1 Development Approach . . . 13
6.2 Tools and Technologies Used . . . 13
6.3 Data Collection Methods . . . 13

7 Implementation **15**
7.1 Frontend Design and Modules . . . 15
7.1.1 Authentication and Session Management . . . 15
7.1.2 Public Feed . . . 15
7.1.3 Post Detail . . . 15
7.1.4 Post Creation Flow . . . 16
7.1.5 Institution Dashboard . . . 16
7.1.6 Signed-In Complaint Help Chatbot . . . 16
7.2 Backend Logic . . . 17
7.2.1 Route Architecture . . . 17
7.2.2 Implemented API Surface . . . 17
7.2.3 Spam Filtering . . . 18
7.2.4 Complaint Help Assistant . . . 18
7.2.5 Database Integration . . . 18
7.3 Code Snippets . . . 18
7.4 Screenshots of the Website . . . 20

8 Results and Impact **21**
8.1 Website Performance . . . 21
8.2 User Value and Community Impact . . . 21
8.3 Before vs. After Comparison . . . 22
8.4 Social Impact Analysis . . . 22

9 Discussion **23**
9.1 Challenges Faced . . . 23

9.2 Limitations of the System 23
9.3 Lessons Learned 23

10 Conclusion and Future Scope **25**
10.1 Summary of Achievements 25
10.2 Possible Improvements 25
10.3 Future Enhancements 26

References **27**

A Survey Questionnaire **29**

B Source Code Snippets **30**

C Appendix Notes for Final Submission **31**

List of Figures

5.1 CivicSignal High-Level Architecture 10
5.2 Entity Relationship Design Summary 11
5.3 Use Case Summary 11
5.4 Post Creation Sequence Summary 12
5.5 Public Feed Wireframe 12
5.6 Institution Dashboard Wireframe 12

7.1 Public Feed Page 20
7.2 New Post Page 20
7.3 Institution Dashboard 20
7.4 Signed-In Complaint Help Chatbot 20

List of Tables

4.1 Hardware Requirements 8
4.2 Software Requirements 9

5.1 Major Entity Groups 11

6.1 Tools and Technologies 13

7.1 Implemented API Routes 17

8.1 Before and After Comparison 22

Chapter 1

Introduction

1.1 Overview of the Community Engagement Project

The Community Engagement Project (CEP) requires students to identify a real-world community problem, engage with stakeholders, and produce a meaningful technical intervention. CivicSignal was developed as a response to a recurring civic challenge: residents often do not know where to report urban issues, do not trust complaint channels, and do not receive transparent progress updates after submitting complaints. The project therefore focuses on building a digital platform that reduces reporting friction while also helping institutions triage and respond more effectively.

1.2 Importance of Digital Solutions for Community Problems

Many civic issues are repetitive, location-specific, and visible to multiple people, yet complaint systems remain siloed across wards, departments, and organizations. A digital solution can unify intake, structure data, reduce ambiguity, preserve audit history, and give citizens a sense of visibility and participation. A web-based platform is especially relevant because it lowers the access barrier for both citizens and institutions and can be used from mobile browsers as well as desktops.

1.3 Objectives of the Website

The main objectives of CivicSignal are:

1. Provide a simple and privacy-aware way for citizens to report civic problems.
2. Support anonymous-by-default participation in the public reporting workflow.

3. Build a ranked public feed that surfaces important issues through community engagement and AI-informed priority.
4. Give NGO staff, government staff, and admins a role-scoped dashboard with actionable summaries and case controls.
5. Integrate AI carefully for spam filtering and in-app complaint guidance without weakening privacy boundaries.

1.4 Problem Statement

Citizens frequently encounter civic issues but lack a trustworthy, transparent, and convenient reporting mechanism. Existing portals are often department-specific, difficult to discover, and weak in public visibility. At the same time, institutional teams receive fragmented or unstructured complaints and spend effort on prioritization, duplication, and moderation. The problem addressed by CivicSignal is the absence of a unified, privacy-aware, community-facing and institution-facing digital system for reporting, validating, and managing civic issues.

1.5 Scope of the Project

The current project scope includes:

- Citizen authentication and profile setup.
- Public feed browsing in guest or signed-in mode.
- Complaint creation with category, language, location, image evidence, and anonymity options.
- Public post detail view with comments, raises, follows, and abuse reporting.
- Role-based institution dashboard for NGO staff, government staff, and admins.
- Groq-based spam filtering and signed-in complaint help chatbot.
- Supabase-backed storage for profile data, posts, media, engagement, and moderation state.

Out of scope for the current submission are native mobile apps, direct integration with government back-office systems, automatic duplicate clustering, and production-grade multilingual UI localization.

Chapter 2

Community Analysis

2.1 Description of the Target Community

The primary target users are urban and semi-urban residents who experience recurring public issues such as sanitation failures, road damage, water access problems, drainage blockages, and safety-related infrastructure faults. The institution-side stakeholders include NGOs working in civic engagement, government staff involved in response operations, and administrators managing moderation and routing.

2.2 Identified Problems

Based on the prior project report, design notes, and current platform model, the key problems are:

- Citizens do not always know the correct channel or department to contact.
- Existing complaint channels can be slow, opaque, or difficult to follow up.
- Public corroboration of issues is weak in traditional complaint systems.
- Fear of identification discourages some users from reporting genuine problems.
- Institutional teams need better triage views, category grouping, and progress tracking.
- Complaint spam, irrelevant content, and vague reports increase triage load.

2.3 Need for a Website-Based Solution

A website-based solution was chosen because it supports:

- direct access from smartphones and desktops,

- a single codebase for citizens and institutions,
- low installation friction,
- compatibility with browser geolocation and media upload,
- straightforward cloud deployment and maintenance.

2.4 Stakeholders Involved

- **Citizens:** submit reports, browse feed, comment, follow, and raise issues.
- **NGO staff:** maintain organization-scoped case progress and internal notes.
- **Government staff:** manage case progress and public workflow movement.
- **Admins:** global moderation, reassignment, and operational oversight.
- **Partner NGO / community organization:** domain collaborator for need identification and process understanding.

Chapter 3

Literature Review

3.1 Existing Websites and Portals in Similar Domains

The provided project report identifies platforms such as FixMyStreet, SeeClickFix, MyGov India, and Swachh City as relevant points of comparison. These systems show the value of digital civic reporting but also reveal gaps around anonymity, community validation, integrated role-based dashboards, and AI-assisted triage.

3.2 Technologies Used in Similar Systems

Comparable systems commonly use:

- React or similar frontend frameworks,
- REST APIs with Express, Rails, Django, or Spring Boot,
- PostgreSQL for relational storage,
- cloud-based file storage for evidence upload,
- mapping and location services,
- increasing use of LLM-based summarization or classification.

3.3 Limitations of Current Solutions

Compared with the current CivicSignal design, common limitations in many existing solutions include:

- weak anonymity support,
- low transparency around complaint progression,

- absence of community corroboration and engagement signals,
- limited NGO-centric workflows,
- little or no built-in AI assistance for intake guidance and spam filtering.

Chapter 4

System Requirements

4.1 Functional Requirements

4.1.1 Citizen Module

- Users can register or log in using email-password or Google OAuth via Supabase.
- Signed-in users can complete a lightweight anonymous profile.
- Guests can browse the public feed and post threads in read-only mode.
- Signed-in users can create issue posts with category, description, language, image uploads, and location mode.
- Signed-in users can comment on, follow, raise, and report posts.
- Signed-in users can access a floating complaint-help chatbot inside the app.

4.1.2 Institution Module

- NGO staff, government staff, and admins can access a dashboard after authentication and authorization.
- The dashboard shows total reports, unresolved issues, high-priority reports, resolved reports, reported items, and average priority.
- The dashboard provides workflow, severity, case-progress, timeline, category, and area hotspot visual summaries.
- NGO staff can update case progress and internal organizational handling status.
- Government staff can additionally update public workflow states.
- Admins can view moderation queues, review reports, and manage broader institutional operations.

4.1.3 AI and Automation Module

- Groq is used to classify post submissions as spam or non-spam before post creation.
- Groq powers a signed-in complaint-help assistant with a restricted domain-specific system prompt.
- The broader enrichment model remains designed for future extension with translation, hazard tags, and severity analysis.

4.2 Non-Functional Requirements

- **Security:** JWT validation through Supabase JWKS; role-based restrictions on private routes.
- **Privacy:** public APIs expose alias snapshot and sanitized area, not exact citizen location.
- **Usability:** mobile-friendly layouts with responsive pages and compact cards.
- **Performance:** feed and summary views optimized for paginated or compact responses.
- **Maintainability:** backend follows route-object pattern; design folder acts as implementation reference.
- **Scalability:** stateless Express API, managed Supabase backend, external AI provider integration.

4.3 Hardware and Software Requirements

Table 4.1: Hardware Requirements

Component	Development	Deployment
CPU	4-core x86-64 or better	2-core cloud instance or better
RAM	8 GB recommended	2–4 GB minimum for app server
Storage	20+ GB local	Cloud storage plus Supabase Storage
Network	Broadband internet	Stable internet for database and Groq API access

Table 4.2: Software Requirements

Component	Version / Detail
Frontend	Next.js 16.1.1, React 19.2.3, Tailwind CSS 4
Backend	Express 4.22.1, TypeScript 5.9.3, Node.js 20 LTS or later
Authentication	Supabase Auth 2.89.0
Database	Supabase PostgreSQL
Storage	Supabase Storage
AI Provider	Groq API using Llama 3.3 70B Versatile
Development Tools	pnpm, tsx, nodemon, VS Code

Chapter 5

System Design

5.1 Architecture Diagram

CivicSignal follows a client-server-cloud architecture:

1. **Client layer:** Next.js frontend for feed, post creation, post detail, auth, institution dashboard, and chatbot widget.
2. **Application layer:** Express API for authentication enforcement, civic data operations, summary generation, moderation actions, spam filtering, and chatbot proxying.
3. **Data layer:** Supabase for authentication, relational data, and media storage.
4. **AI layer:** Groq for spam filtering and in-app complaint help assistant.

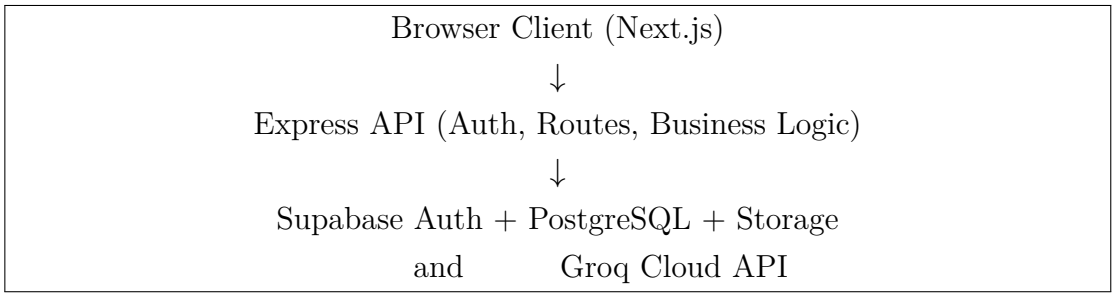

Figure 5.1: CivicSignal High-Level Architecture

5.2 Database Design

The current database model, as documented in `design/dbDesign.md`, contains core reporting, identity, geography, engagement, moderation, AI assessment, and summary entities. Major tables include `profiles`, `posts`, `post_media`, `post_comments`, `post_raises`,

`post_follows`, `post_reports`, `post_status_history`, `institution_case_views`, and `daily_post_summaries`.

Table 5.1: Major Entity Groups

Group	Main Tables
Identity	`profiles`, `institution_organizations`
Geography	`areas`
Taxonomy	`categories`
Core Reporting	`posts`, `post_media`
AI Layer	`post_ai_assessments`
Engagement	`post_comments`, `post_raises`, `post_follows`
Moderation	`post_reports`
Audit / Triage	`post_status_history`, `institution_case_views`
Analytics	`daily_post_summaries`

Refer to `design/dbDesign.md` for full ER model.
Core relationship summary: Profiles author posts; posts have media, AI assessments, comments, raises, follows, reports, and status history; institution case views connect posts with responder organizations; daily summaries aggregate posts by date, area, category, workflow, and severity.

Figure 5.2: Entity Relationship Design Summary

5.3 UML Diagrams

5.3.1 Use Case View

The main actors are citizen, NGO staff, government staff, and admin. Key use cases include register/login, browse feed, create complaint, view thread, comment, follow, raise, report, view dashboard, update case status, update workflow status, and moderate reported content.

Actors: Citizen, NGO Staff, Government Staff, Admin
Citizen: register, login, browse feed, open thread, create complaint, comment, follow, raise, report, use chatbot
NGO Staff: dashboard access, update case progress, add notes
Government Staff: NGO actions + update public workflow
Admin: all institution actions + moderation review and global oversight

Figure 5.3: Use Case Summary

5.3.2 Sequence View for Post Creation

The post creation sequence is:

1. Client loads categories and areas.
2. User fills complaint form and optionally uploads images.
3. Client uploads images to Supabase Storage.
4. Client submits complaint payload to `POST /posts`.
5. Backend calls Groq spam classifier.
6. If accepted, backend inserts post and related metadata.
7. Client redirects user to the post detail page.

Citizen → Next.js Client → Supabase Storage
Citizen → Express API → Groq Spam Filter
Express API → Supabase PostgreSQL
API Response → Post Detail Page

Figure 5.4: Post Creation Sequence Summary

5.4 Wireframes and UI Design

The application uses a unified visual language built with Tailwind CSS. The feed uses a social-card style layout, the post composer uses a guided form layout, and the institution dashboard uses metric cards, distribution cards, filters, and work queues.

Wireframe Placeholder: Public Feed
Hero banner, search/filter sidebar, ranked post cards, engagement actions, signed-in floating chatbot icon.

Figure 5.5: Public Feed Wireframe

Wireframe Placeholder: Institution Dashboard
KPI cards, workflow/severity/case charts, category and area hotspot panels, focused queue, moderation queue.

Figure 5.6: Institution Dashboard Wireframe

Chapter 6

Methodology

6.1 Development Approach

The project was developed incrementally. The team first established design documents for features, constraints, API contracts, and the database model. Implementation then progressed through authentication, feed delivery, post creation, detail view, institution workflows, AI integrations, and reporting documentation.

6.2 Tools and Technologies Used

Table 6.1: Tools and Technologies

Category	Technology
Frontend Framework	Next.js 16.1.1, React 19.2.3
Styling	Tailwind CSS 4
Backend Framework	Express 4.22.1 with TypeScript
Authentication	Supabase Auth, email-password login, Google OAuth
Database	Supabase PostgreSQL
Storage	Supabase Storage
AI Services	Groq chat completions for spam filter and complaint help assistant
API Design	Route-object pattern plus OpenAPI design file
Runtime Tooling	Node.js, pnpm, tsx, nodemon
Version Control	Git and GitHub

6.3 Data Collection Methods

The original project report and community-analysis material indicate three main inputs:

- community survey and observation,
- NGO interaction and workflow discussion,
- secondary research on comparable civic-tech platforms and technical stacks.

Chapter 7

Implementation

7.1 Frontend Design and Modules

7.1.1 Authentication and Session Management

The frontend uses an `AuthProvider` context built on top of Supabase Auth. It fetches the active session, listens to auth state changes, refreshes profile data through `/me`, and exposes session plus profile state to the rest of the application. Login and registration are implemented through a shared `AuthScreen` component and support both email-password authentication and Google OAuth.

7.1.2 Public Feed

The public feed page:

- fetches ranked or sorted posts from `/feed`,
- supports search by text, category, or area,
- supports workflow filters,
- lets signed-in users follow an issue,
- links to post detail threads,
- shows guest-mode messaging for unauthenticated users.

7.1.3 Post Detail

The post detail page fetches a single post and its comments, and supports:

- adding comments,
- toggling raise,

- toggling follow,
- reporting posts,
- reporting inappropriate images through abuse notes,
- viewing AI summary and media evidence.

7.1.4 Post Creation Flow

The new-post page implements:

- category and source-language selection,
- description entry,
- image validation and upload to Supabase Storage,
- manual, none, or auto-detected location capture,
- anonymous posting toggle,
- backend post submission with Groq-based spam screening.

7.1.5 Institution Dashboard

The institution dashboard now includes:

- KPI cards for total, unresolved, high-priority, resolved, reported, and average priority values,
- workflow mix, severity breakdown, and case-progress distribution cards,
- daily intake chart,
- top categories and area hotspots,
- filterable focused queue by status, severity, category, queue type, search, and sort,
- role-aware case actions,
- admin moderation queue.

7.1.6 Signed-In Complaint Help Chatbot

The app header includes a floating chat widget visible only to signed-in users. The widget sends user messages to the backend `/assistant/chat` endpoint, which forwards them to Groq with a tightly restricted context prompt. The assistant is constrained to answer only complaint-related, portal-related, and UrbanFix usage questions in English.

7.2 Backend Logic

7.2.1 Route Architecture

The backend dynamically discovers route modules and applies middleware based on a route-object contract. Each route declares method, path, auth level, rate limit, key type, and handler. This keeps the backend consistent and explicit.

7.2.2 Implemented API Surface

Table 7.1: Implemented API Routes

Method	Path	Purpose
GET	`/health`	Base health endpoint
GET	`/health/groq`	Groq integration health endpoint
GET	`/me`	Authenticated profile hydration
POST	`/profiles/register`	Create app profile row
GET	`/feed`	Public feed retrieval
GET	`/categories`	Category lookup for signed-in users
GET	`/areas`	Area lookup for signed-in users
POST	`/posts`	Create a civic issue post
GET	`/posts/:postId`	Public post detail
GET	`/posts/:postId/comments`	Comment list
POST	`/posts/:postId/comments`	Create comment
POST	`/posts/:postId/raises`	Toggle raise
POST	`/posts/:postId/follows`	Toggle follow
POST	`/posts/:postId/reports`	Report abuse or invalid content
GET	`/institution/posts/:postId`	Institution-only post detail
PATCH	`/institution/posts/:postId`	Institution update endpoint
GET	`/institution/summaries/overview`	Dashboard summary endpoint
GET	`/institution/summaries/areas/:areaId`	Area summary endpoint
POST	`/assistant/chat`	Signed-in complaint assistant endpoint

7.2.3 Spam Filtering

Before post insertion, the backend sends a narrow classification prompt to Groq and expects only `SPAM` or `NOT_SPAM`. If the model returns spam or an invalid result, the request is rejected. This keeps the public feed cleaner and reduces non-genuine submissions.

7.2.4 Complaint Help Assistant

The assistant endpoint sanitizes chat history, constrains message length, and sends the user conversation to Groq with a domain-specific system prompt. This prompt defines the app context, the feed and new-post flows, and hard refusal boundaries for off-topic use.

7.2.5 Database Integration

The backend uses the Supabase service-role client for application-managed queries and enforces privacy at the application layer. Public data excludes sensitive fields such as exact coordinates, while institution endpoints may include private operational data after role checks.

7.3 Code Snippets

Route Object Pattern

Listing 7.1: Express route-object pattern used in the backend

```
const assistantRouter: RouterObject = {
  path: "/assistant",
  functions: [
    {
      method: "post",
      props: "/chat",
      authorization: "required",
      rateLimit: "gameplay",
      keyType: "user",
      handler: async (req, res) => {
        const reply = await getComplaintAssistantReply(req.body);
        res.status(200).json(reply);
      },
    },
  ],
};
```

Groq Spam Classification Call

Listing 7.2: Groq-based spam filtering request pattern

```
const response = await fetch(`${ENV.GROQ_BASE_URL}/chat/completions`, {
  method: "POST",
  headers: {
    Authorization: `Bearer ${ENV.GROQ_KEY}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    model: ENV.GROQ_MODEL,
    messages: [
      { role: "system", content: buildSystemPrompt() },
      { role: "user", content: buildPrompt(input) },
    ],
    temperature: 0,
    max_completion_tokens: ENV.GROQ_MAX_TOKENS,
  }),
});
```

Groq Complaint Assistant Call

Listing 7.3: Complaint help assistant request to Groq

```
const response = await fetch(`${ENV.GROQ_BASE_URL}/chat/completions`, {
  method: "POST",
  headers: {
    Authorization: `Bearer ${ENV.GROQ_KEY}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    model: ENV.GROQ_CHAT_MODEL,
    temperature: 0.2,
    max_completion_tokens: ENV.GROQ_CHAT_MAX_TOKENS,
    messages: [
      { role: "system", content: ASSISTANT_SYSTEM_PROMPT },
      ...history,
      { role: "user", content: message },
    ],
  }),
});
```

7.4 Screenshots of the Website

Insert screenshot here: Public feed page with hero section, filters, and ranked post cards.

Figure 7.1: Public Feed Page

Insert screenshot here: New post page showing category, description, image upload, and location controls.

Figure 7.2: New Post Page

Insert screenshot here: Institution dashboard with KPI cards, charts, and filtered work queue.

Figure 7.3: Institution Dashboard

Insert screenshot here: Floating Groq complaint-help chatbot opened for a signed-in user.

Figure 7.4: Signed-In Complaint Help Chatbot

Chapter 8

Results and Impact

8.1 Website Performance

Formal load-test tables have not yet been regenerated for the newest Groq-based architecture, but the current codebase compiles successfully in both client and server builds. Feed, authentication, post creation, and dashboard workflows are implemented end-to-end. The architecture is suitable for continued benchmarking after deployment.

8.2 User Value and Community Impact

CivicSignal creates value in the following ways:

- citizens gain a clearer and lower-friction path to file local complaints,
- anonymity lowers hesitation to report sensitive local problems,
- public feed visibility creates community validation and transparency,
- institutions receive structured queues instead of scattered complaint intake,
- AI reduces spam and offers contextual guidance inside the application.

8.3 Before vs. After Comparison

Table 8.1: Before and After Comparison

Before CivicSignal	After CivicSignal
Citizens rely on fragmented manual complaint channels	Citizens can create structured digital complaints in one platform
No public corroboration of issue urgency	Raises, follows, comments, and ranked visibility show broader impact
Identity disclosure discourages reporting	Anonymous-by-default profile and post design reduce that barrier
Institutions lack summarized operational dashboards	Dashboard provides KPIs, charts, hotspots, and filtered work queues
Complaint noise can overwhelm triage	Groq-based spam filter screens post submissions at ingestion

8.4 Social Impact Analysis

The platform aligns with broader social outcomes in transparency, accountability, and civic participation. It supports public visibility of local issues while preserving sensitive identity-linked information. It also demonstrates how AI can be introduced in a bounded and socially useful way: for noise reduction and guided civic assistance rather than for replacing human institutional judgment.

Chapter 9

Discussion

9.1 Challenges Faced

- Designing anonymity without losing accountability in engagement actions.
- Maintaining different role capabilities in one shared application.
- Handling provider-specific AI failures such as 402 and 429 responses before stabilizing on Groq.
- Building a dashboard that is both professional and actionable for institutions.
- Keeping report data useful for institutions while respecting public privacy constraints.

9.2 Limitations of the System

- The project currently depends on external Groq API availability for AI features.
- Screenshots and production deployment metrics still need to be finalized for the final printed report.
- Multilingual UI support is not yet fully implemented across all pages.
- Integration testing and CI automation remain limited.
- Government portal integration is advisory only; the chatbot does not submit complaints externally.

9.3 Lessons Learned

- Strong design documentation reduces rework during implementation.

- Role-scoped workflows must be reflected consistently in both UI and backend authorization.
- AI integration needs fallback strategies because provider behavior and quotas can change.
- Complaint systems require both citizen usability and institutional operability; solving one side alone is not enough.

Chapter 10

Conclusion and Future Scope

10.1 Summary of Achievements

CivicSignal now includes:

- working citizen authentication and anonymous profile flows,
- public feed with filters and sorting,
- complaint creation with media and location handling,
- post detail with engagement and reporting actions,
- role-aware institution dashboard with charts and operational queues,
- Groq-based spam filtering,
- signed-in complaint-help chatbot with restricted domain guidance,
- detailed design documentation for database, API, constraints, and user flow.

10.2 Possible Improvements

- add automated tests for key user and institution flows,
- improve dashboard exports and analytics views,
- add screenshot assets and final field validation for print submission,
- activate more database-level security policies where appropriate,
- add explicit deployment and CI documentation.

10.3 Future Enhancements

- multilingual chatbot and UI,
- mobile application or PWA enhancements,
- duplicate issue clustering and similarity detection,
- GIS heatmaps and locality trend exploration,
- richer institutional audit trails and notifications,
- optional integration guidance modules for city-specific external grievance portals.

References

Bibliography

[1] mySociety, "FixMyStreet," https://www.fixmystreet.com

[2] SeeClickFix, "SeeClickFix Platform," https://seeclickfix.com

[3] Supabase, "Supabase Documentation," https://supabase.com/docs

[4] Vercel, "Next.js Documentation," https://nextjs.org/docs

[5] Express.js, "Express Documentation," https://expressjs.com

[6] Groq, "Groq Documentation," https://console.groq.com/docs

[7] OpenAPI Initiative, "OpenAPI Specification," https://spec.openapis.org

[8] Ramdeobaba University, "Guidelines for Project Report Preparation," provided PDF, 2026.

[9] CivicSignal Team, "CivicSignal Project Report," provided PDF, 2026.

Appendix A

Survey Questionnaire

The following survey items may be included as the questionnaire appendix:

1. How often do you encounter civic problems in your locality?
2. Have you ever reported a civic issue formally?
3. If not, what stopped you from reporting it?
4. Do you use a smartphone with internet access?
5. Which language would you prefer in a complaint platform?
6. Would anonymity increase your willingness to report?
7. Which issue categories are most common in your area?
8. Would you use a mobile-friendly website for civic complaints?

Appendix B

Source Code Snippets

Complaint Assistant System Prompt Excerpt

```
You help users inside UrbanFix app.
Answer only complaint, reporting, grievance routing,
UrbanFix usage, and official portal guidance questions.
Reply in English.
```

Frontend Chat Widget Trigger

```
if (loading || !session) {
  return null;
}
```

Appendix C

Appendix Notes for Final Submission

Before final submission, update the following placeholders in this report:

- guide name and signatures,
- NGO name and signed acknowledgement copy,
- department-approved title page details,
- final screenshots of implemented pages,
- any validated field-study numbers or feedback statistics,
- final project PDF filename per university naming rules.